

机器学习上机实验13-14：神经网络-3

实验题目：Transformer Encoder 图像分类实现

一、实验目的

本实验基于 PyTorch 从零搭建一个 Transformer 图像分类模型，并完成 MNIST 手写数字分类任务。
通过本实验，掌握：

- Transformer Encoder结构
- Position Encoding作用
- Multi-Head Attention机制
- Transformer分类流程
- PyTorch Transformer模块使用方法

二、实验环境与设定

- Python 3.10+
- PyTorch
- CPU/GPU版本实现皆可
- 数据集：MNIST

三、实验背景

MNIST数据集中的每张图片大小为：

代码块

```
1 28 × 28
```

本实验将每一行像素视为一个Token：

代码块

```
1  第1行 → Token1
2  第2行 → Token2
3  ...
4  第28行 → Token28
```

因此：

代码块

```
1  输入图像
2  28 × 28
3  ↓
4  序列长度：
5  28
6  ↓
7  Token维度：
8  28
```

Transformer输入：

代码块

```
1  (batch_size, 28, 28)
```

四、网络结构

本实验使用如下网络：

代码块

```
1  输入图像
2  (batch,1,28,28)
3  ↓
4  reshape
5  (batch,28,28)
6  ↓
7  Linear Embedding
8  28 → 64
9  ↓
10 Position Encoding
11 ↓
12 Transformer Encoder × 2
```

```
13 ↓
14 Mean Pooling
15 ↓
16 Linear
17 64 → 10
18 ↓
19 分类结果
```

五、实验任务

1. 数据加载

导入MNIST数据集，并执行标准化。

要求：

代码块

```
1 batch_size = 128 # 或者是硬件配置可接受的batch size
```

数据预处理：

代码块

```
1 ToTensor()
2 Normalize((0.5,), (0.5,))
```

2. 构建Transformer分类器

建立如下类：

代码块

```
1 class TransformerClassifier(nn.Module):
```

- Step 1: Embedding层

建立：

代码块

```
1 self.embedding
```

作用：

代码块

```
1 28维像素
2 ↓
3 64维特征
```

要求：

代码块

```
1 nn.Linear(28,64)
```

输入：

代码块

```
1 (batch,28,28)
```

输出：

代码块

```
1 (batch,28,64)
```

- Step 2: Position Encoding

建立可学习位置编码：

代码块

```
1 self.pos_embedding
```

大小：

代码块 1, 28, 64)

要求：

代码块

```
1 nn.Parameter(...)
```

前向传播，实现：

代码块

```
1 x = x + self.pos_embedding
```

思考：

为什么要加位置编码？

- Step 3: Transformer Encoder

建立Encoder Layer：

代码块

```
1 nn.TransformerEncoderLayer(  
2     d_model=64,  
3     nhead=4,  
4     dim_feedforward=128,  
5     batch_first=True  
6 )
```

建立：

代码块

```
1 Transformer Encoder
```

层数：

代码块

1 2层

输入维度：

代码块

```
1 (batch,28,64)
```

输出维度：

代码块

```
1 (batch,28,64)
```

- Step 4: 分类头

使用：

代码块

```
1 x.mean(dim=1)
```

完成：

代码块

```
1 28个Token特征
```

```
2 ↓
```

```
3 全局特征
```

输出：

代码块

```
1 (batch,64)
```

最后：

代码块

```
1 Linear(64,10)
```

得到：

代码块

```
1 (batch,10)
```

3. 完成Forward函数

按照以下顺序实现：

代码块

```
1  输入图片
2  ↓
3  去掉通道维
4  ↓
5  Embedding
6  ↓
7  Position Encoding
8  ↓
9  Transformer
10 ↓
11 Mean Pooling
12 ↓
13 分类器
```

格式如下：

代码块

```
1 def forward(self,x):
2
3     x = _____
4     ...
5     x = _____
6
7     return x
```

4. 模型训练

训练参数：

代码块

```
1  Epoch = 3
2  Batch Size = 128 # 或者是硬件配置可接受的batch size
3  Learning Rate = 0.001
```

优化器：

代码块

```
1  Adam
```

损失函数：

代码块

```
1  CrossEntropyLoss
```

每个Epoch输出：

代码块

```
1  Epoch
2  Train Loss
3  Test Accuracy
```

示例：

代码块

```
1  Epoch 1
2  Loss: 0.45
3  Acc: 89.5%
```

5. 查看模型结构

训练前：

代码块

```
1 print(model)
```

回答：

模型中共有几个Transformer Encoder层？

每个Token最终维度是多少？

6. Head数量实验

修改：

代码块

```
1 nhead
```

分别测试：

代码块

```
1 1
2 2
3 4
4 8
```

训练：

代码块

```
1 1 Epoch
```

记录：

Head数	Accuracy
1	
2	
4	
8	

回答：

Head数增加后性能是否一定提升？为什么？

7. Position Encoding (PE) 消融实验

实验A：

代码块

```
1 x = x + self.pos_embedding
```

实验B：

代码块

```
1 不加Position Encoding
```

记录：

模型	Accuracy
With PE	
Without PE	

六、附加挑战（感兴趣同学可尝试实现，不影响分数）

尝试修改为使用特殊token进行分类：

代码块

```
1 Mean Pooling
2 ↓
3 CLS Token
```

七、作业要求

- **代码：**整理出代码文件，规范添加注释。
- **报告：**依据本实验内容，形成实验报告。